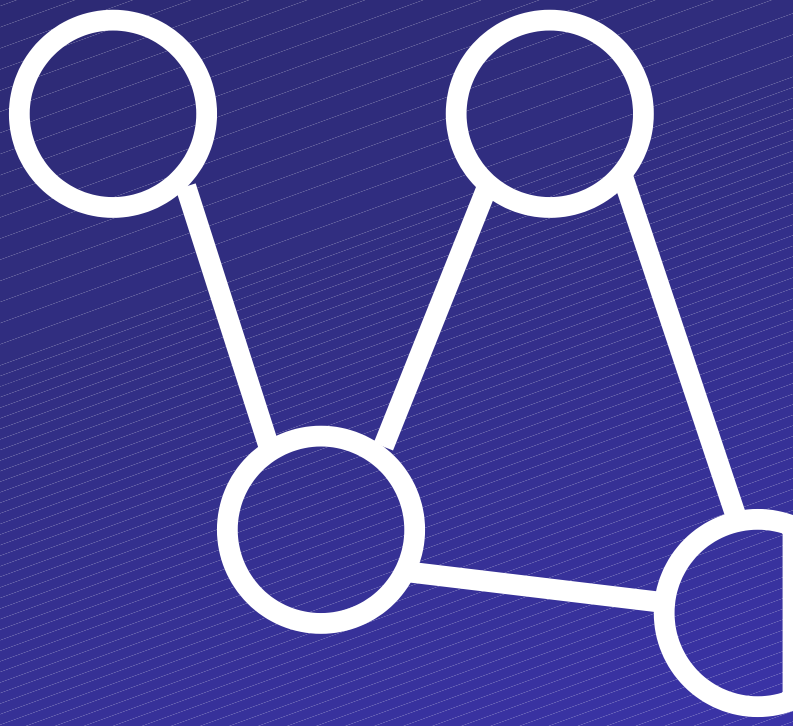
MEMORIA TÉCNICA

Redes Bayesianas

Diseño, implementación y validación de una aplicación web de inferencia exacta con eliminación de variables.



Facultad de Ingeniería Eléctrica · UMSNH
Curso: Modelos probabilistas
Luis Adrián Gómez Saavedra · Luis Roberto Villafuerte Gómez · Jonathan Miguel Anguiano Huitron
Noviembre de 2026
https://github.com/2201146h-AdrianGomez/ProyectoModelosProbabilistas

Contenido

1. Introducción **4**

1.1 Objetivos 4

1.2 Equipo y repositorio 4

1.3 Cómo está organizado este documento 4

2. Requisitos y cumplimiento **5**

3. Marco teórico **6**

3.1 Redes bayesianas 6

3.2 Inferencia: qué se calcula 6

3.3 Eliminación de variables 6

3.4 Orden de eliminación y costo 7

3.5 Verificación independiente 8

4. Diseño y arquitectura **9**

4.1 Decisiones principales 9

4.2 Estructura de módulos 10

4.3 Estructuras de datos 10

4.4 Tamaño del código 10

5. Parte 1 · Núcleo matemático **11**

5.1 Modelo y validaciones (red.js) 11

5.2 Eliminación de variables (inferencia.js) 11

5.3 Enumeración de referencia 11

5.4 Una consulta de principio a fin 12

6. Parte 2 · Interfaz **13**

6.1 Estructura y estilos 13

6.2 Estado global y barra superior (main.js) 13

6.3 Avisos y eventos (util.js) 13

6.4 Dibujo del grafo (grafo.js) 13

6.5 Editor (editor.js) 13

7. Parte 3 · Consultas, archivos y salidas **16**

7.1 Panel de consultas (consulta.js) 16

7.2 Guardar y abrir (almacenamiento.js) 16

7.3 Informe imprimible (informe.js) 16

7.4 Ejemplos y herramienta generadora 17

8. Validación y pruebas 18
8.1 Estrategia 18
8.2 Pruebas automáticas 18
8.3 Resultados de los cinco ejemplos 18
8.4 Pruebas con la interfaz 19
9. Los cinco ejemplos 20
9.1 Qué aporta cada ejemplo 20
10. Reparto de trabajo y contribuciones 22
10.1 Reparto por partes 22
10.2 Orden de subida a GitHub 22
10.3 Evidencia de contribuciones 23
10.4 Qué debe poder explicar cada integrante 23
11. Limitaciones y trabajo futuro 24
12. Conclusiones 24
Referencias 25
Anexo A. Salida de las pruebas automáticas 26
Anexo B. Estructura del repositorio 27
Anexo C. Evidencias 28

1. Introducción

Este documento es la memoria técnica del proyecto del curso de **Modelos probabilistas** Facultad de Ingeniería Eléctrica, UMSNH . El proyecto consiste en una **aplicación web** para construir, visualizar, consultar y resolver **redes bayesianas** mediante el algoritmo de **eliminación de variables**. Aquí se explica qué se hizo, cómo está organizado el código, por qué se tomaron las decisiones principales, cómo se comprobó que los resultados son correctos y quién hizo cada parte.

1.1 Objetivos

- Permitir crear redes de 3 a 10 nodos con estados, relaciones y tablas de probabilidad condicional CPT , validando que el grafo sea acíclico y que cada fila de cada CPT sume 1.
- Resolver consultas P(consulta | evidencia) con eliminación de variables, mostrando los pasos.
- Guardar y abrir redes en archivos, y producir una salida imprimible.
- Entregar un repositorio documentado, con ejemplos, pruebas y evidencia de las contribuciones de cada integrante.

1.2 Equipo y repositorio

Integrante	Parte asignada
Luis Roberto Villafuerte Gómez	Parte 1 · Núcleo matemático
Luis Adrián Gómez Saavedra	Parte 2 · Interfaz
Jonathan Miguel Anguiano Huitron	Parte 3 · Consultas, archivos y salidas

Repositorio: https://github.com/2201146h-AdrianGomez/ProyectoModelosProbabilistas

1.3 Cómo está organizado este documento

La sección 2 resume el cumplimiento de los requisitos. La 3 explica la teoría necesaria. Las secciones 4 a 7 describen el diseño y cada una de las tres partes. La 8 presenta las pruebas, la 9 los cinco ejemplos y la 10 el reparto de trabajo. Las últimas secciones cubren el uso de herramientas de IA, las limitaciones y las conclusiones.

2. Requisitos y cumplimiento

Requisito	Cómo se cumple	Dónde
Aplicación web en JavaScript, HTML y CSS, publicable en GitHub Pages o Netlify	Páginas estáticas sin dependencias ni paso de compilación; funciona incluso abriendo `index.html` directamente (sin servidor).	`index.html`, `css/`, `js/`
Redes de 3 a 10 nodos con nombre, estados y padres	Límites en `Red.MIN_NODOS/MAX_NODOS`; de 2 a 8 estados por nodo.	`red.js`, `editor.js`
Validar que sea un DAG	La interfaz deshabilita padres que crearían ciclo; el validador además detecta ciclos en archivos y reporta la ruta A B A.	`red.js`
CPT por nodo; cada fila suma 1	Edición en tabla con suma por fila, normalización y tolerancia de 10^{-6}.	`red.js`, `editor.js`
Visualizar la red	Grafo dirigido en SVG con nodos arrastrables y acomodo automático.	`grafo.js`
Consultas con evidencia, resueltas con eliminación de variables	Una o varias variables de consulta; evidencia arbitraria; cuatro formas de elegir el orden de eliminación.	`inferencia.js`, `consulta.js`
Mostrar el procedimiento	Registro de pasos (factores, restricciones, cada τ) visible en pantalla.	`consulta.js`
Guardar, abrir y modificar redes	Formato JSON documentado y validado al abrir; borrador automático en el navegador.	`almacenamiento.js`, `docs/FORMATO.md`
Salida impresa	Informe imprimible (red, tablas, resultado, pasos) y descarga HTML.	`informe.js`
Cinco ejemplos 3, 4, 5, 6 y 10 nodos)	Cinco redes con consultas resueltas y verificadas de tres maneras.	`ejemplos/`
Repositorio con README, documentación, ejemplos, pruebas y evidencia	Ver estructura en el anexo B.	repositorio
Manual de usuario y memoria técnica en PDF	Este documento y el manual de usuario.	`docs/`

3. Marco teórico

3.1 Redes bayesianas

Una red bayesiana es un grafo dirigido acíclico en el que cada nodo es una variable aleatoria discreta y cada nodo tiene una tabla de probabilidad condicional CPT que da P X | padres(X)). Las flechas expresan dependencia directa; la ausencia de flecha expresa una independencia condicional. La propiedad central es que la distribución conjunta se **factoriza** como el producto de las CPT Pearl, 1988; Koller y Friedman, 2009

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{padres}(X_i))$$

Una tabla completa de la conjunta necesitaría $\prod_i |X_i|$ números (por ejemplo 2^{10} 1024 para diez variables binarias); las CPT necesitan solo los de cada nodo con sus padres.

3.2 Inferencia: qué se calcula

Dada una consulta Q y una evidencia e (valores observados de ciertas variables), se quiere P Q | e). Sea H el conjunto de variables ocultas (ni consulta ni evidencia). Por definición de probabilidad condicional y marginalización:

$$P(Q \mid e) = \frac{P(Q, e)}{P(e)} = \frac{\sum_{h \in H} \prod_i P(X_i \mid \text{padres}(X_i))\Big|_{Q, e, h}}{P(e)}$$

El numerador se calcula para cada valor de Q y el denominador P(e) es la suma de todos ellos; por eso basta calcular el numerador y **normalizar**. La suma directa (**enumeración**) recorre todas las combinaciones de las variables ocultas, lo cual es exponencial en su número.

3.3 Eliminación de variables

La eliminación de variables Zhang y Poole, 1994; Dechter, 1999 aprovecha que el producto de factores se puede reorganizar: una variable que aparece solo en algunos factores se puede sumar *antes* de multiplicar el resto. Se trabaja con **factores**, funciones que asignan un número a cada combinación de valores de un conjunto de variables, y con tres operaciones:

- **Restringir** un factor por la evidencia X x: se conservan solo las filas con X x y se quita la variable X.
- **Multiplicar** factores: el resultado tiene como variables la unión de las de ambos, y cada valor es el producto de los valores correspondientes.
- **Sumar una variable** v de un factor: se suman los valores sobre todos los estados de v, y v desaparece del factor.

Eliminar la variable v consiste en: 1 multiplicar todos los factores que contienen a v, obteniendo un factor ϕ; 2 sumar v de ϕ, obteniendo τ, que sustituye a los factores multiplicados. Cuando ya no quedan variables ocultas, se multiplican los factores que sobran (que solo mencionan variables de consulta), la suma de todos sus valores es P(e) y al dividir se obtiene la distribución buscada.

```
eliminacionDeVariables(red, consulta, evidencia, heurística):
    F ← { factorDeNodo(n) : n ∈ nodos(red) }            // una CPT por nodo
    para cada (X = x) en evidencia:
        F ← { restringir(f, X, x) : f ∈ F }
    H ← variables que no están en consulta ni en evidencia
    mientras H ≠ ∅:
        v ← elegirVariable(H, F, heurística)
        G ← { f ∈ F : v ∈ vars(f) }                     // factores que mencionan a v
        φ ← multiplicar(G)
        τ ← sumarVariable(φ, v)
        F ← (F \ G) ∪ { τ };   H ← H \ { v }
    φ ← multiplicar(F);  reordenar φ a las variables de consulta
    Z ← suma de los valores de φ                        // = P(evidencia)
    devolver φ / Z, Z
```

Ejemplo guiado

Para el Ejemplo 3, P(Robo | Juan = Sí, María = Sí): tras restringir, quedan los factores P Robo), P Terremoto), P(Alarma | Robo, Terremoto), P(Juan Sí | Alarma) y P(María Sí | Alarma). Se elimina primero *Terremoto* (se multiplican P Terremoto) y P(Alarma | Robo, Terremoto) y se suma sobre Terremoto, quedando τ_1(Robo, Alarma)) y luego *Alarma* (τ_2(Robo)). Lo que queda es P Robo) · τ_2(Robo). El cálculo numérico completo está en el manual de usuario, sección 7.3

Robo	P(Robo)·τ₂(Robo)	Normalizado
Sí	0.000592243	0.284172
No	0.00149186	0.715828
Suma P(evidencia)	0.0020841	1.000000

3.4 Orden de eliminación y costo

El resultado no depende del orden, pero el costo sí. Al eliminar v se construye un factor ϕ cuyo tamaño es el producto de los estados de todas las variables que aparecen en los factores multiplicados; el trabajo total es del orden del mayor de esos tamaños, que es exponencial en el «ancho inducido» del orden elegido. Encontrar el mejor orden es un problema NP-difícil Arnborg, Corneil y Proskurowski, 1987 , y la inferencia exacta en general también lo es Cooper, 1990 . En la práctica se usan **heurísticas voraces**:

- **Mínimo relleno (*min-fill*)**: elegir la variable cuya eliminación crea menos conexiones nuevas entre sus vecinas en el grafo de interacción (las variables que comparten algún factor). Es la heurística predeterminada.
- **Mínimo grado**: elegir la variable con menos vecinas.

La aplicación además ofrece «orden de la lista de nodos» y un orden personalizado, para comparar. En todos los casos los desempates se resuelven por posición en la lista de nodos, de modo que el resultado de la aplicación es determinista.

3.5 Verificación independiente

Como referencia se implementó la **enumeración exhaustiva**: se recorren todas las asignaciones de las variables de la red, se descartan las que contradicen la evidencia y se acumula el producto de CPT en el valor de consulta correspondiente. Es lenta pero casi no puede equivocarse, y no comparte código con la eliminación (no usa factores), por lo que sirve como «segunda opinión».

4. Diseño y arquitectura

4.1 Decisiones principales

Decisión	Motivo
JavaScript simple, sin bibliotecas ni empaquetador	Que el repositorio se pueda abrir con doble clic, publicar en GitHub Pages o Netlify sin configuración, y que cada integrante pueda leer todo su código.
Módulos como funciones autoejecutables sobre un espacio de nombres global `RB`	Los módulos ES requieren servidor (no funcionan con `file://`). El mismo código del núcleo corre en el navegador y en Node para las pruebas.
Núcleo separado de la interfaz	`red.js` e `inferencia.js` no tocan el DOM se pueden probar solos y repartir como una parte independiente.
Comunicación por eventos	Los paneles no se llaman entre sí: cuando algo cambia emiten un evento y los demás se redibujan. Así se evitan dependencias circulares entre las partes de distintas personas.
Nombres de código en español	Pedido por el equipo; facilita explicar el código en el curso.
Verificar cada consulta con otro método	La aplicación confirma en pantalla que su respuesta coincide con la enumeración.
Formato de archivo propio en JSON	Legible, editable a mano y fácil de validar; documentado en `docs/FORMATO.md`.

4.2 Estructura de módulos

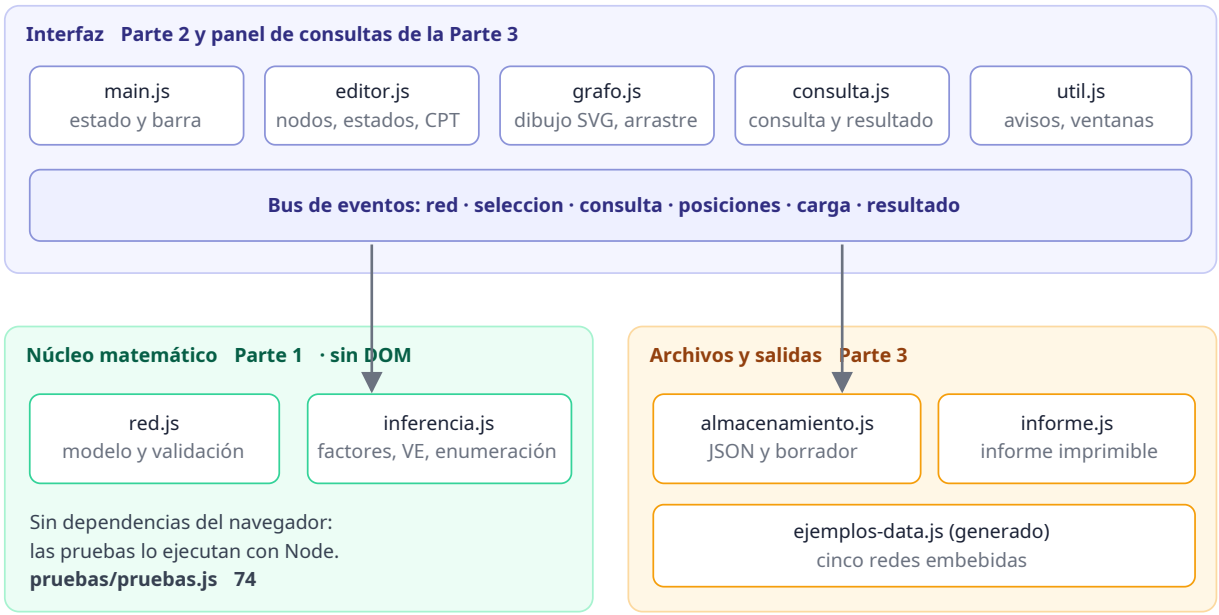


Figura 4.1. Módulos de la aplicación y su relación. La interfaz usa al núcleo; el núcleo no conoce a la interfaz.

4.3 Estructuras de datos

Red. Objeto con `nombre`, `descripcion` y una lista `nodos`. Cada nodo tiene `nombre`, `estados` (lista de textos), `padres` (lista de nombres), `cpt` (matriz con una fila por combinación de estados de los padres y una columna por estado) y `posicion` (x, y). El orden de las filas es el de un contador en el que el primer padre cambia más lento y el último más rápido.

Factor. Objeto `{ vars, card, estados, valores, etiqueta }`: la lista de variables, cuántos estados tiene cada una, los nombres de los estados y un arreglo plano de números con la misma convención de orden (la primera variable cambia más lento). Con esa convención, el índice de una fila se calcula como en un número de base mixta, y multiplicar o sumar se reduce a recorrer índices.

Resultado de la inferencia. Incluye la distribución, P(evidencia), el orden de eliminación, la heurística y la lista de pasos (`inicio`, `evidencia`, `eliminacion`, `final`), de la que la interfaz y el informe toman el texto del «paso a paso».

4.4 Tamaño del código

El proyecto tiene unas 5 350 líneas en total: 1 265 en la Parte 1 (núcleo y pruebas), 2 384 en la Parte 2 (de las cuales 1 351 son hojas de estilo) y 1 699 en la Parte 3 (de las cuales 786 son el archivo de ejemplos, generado automáticamente). El anexo B lista los archivos.

5. Parte 1 · Núcleo matemático

Archivos: `js/nucleo/red.js`, `js/nucleo/inferencia.js` y `pruebas/pruebas.js`.

5.1 Modelo y validaciones (`red.js`)

- **Constantes:** 3 a 10 nodos, 2 a 8 estados, tolerancia 10^{-6} para que una fila sume 1.
- **Operaciones de edición:** crear red, agregar y borrar nodo, renombrar nodo (actualiza a los hijos), cambiar estados (reinicia las tablas afectadas y devuelve la lista de nodos afectados para avisar al usuario), renombrar estado (conserva las tablas), cambiar padres.
- **Detección de ciclos:** `encontrarCiclo` hace una búsqueda en profundidad con tres colores (sin visitar, en curso, terminado); una arista a un nodo «en curso» revela un ciclo y se reconstruye la ruta para mostrarla. `descendientes` permite a la interfaz deshabilitar los padres que crearían un ciclo.
- **Validación completa** (`validar`): cantidad de nodos, nombres únicos sin distinguir mayúsculas, estados distintos, padres existentes, sin repetir y distintos del propio nodo, ausencia de ciclos y, para cada CPT, número correcto de filas y columnas, valores entre 0 y 1 y suma por fila. Devuelve la **lista completa** de errores (no solo el primero) con texto en español.
- **Acomodo automático** (`autoLayout`): coloca los nodos por niveles según su profundidad en el grafo.

5.2 Eliminación de variables (`inferencia.js`)

Funciones principales: `factorDeNodo` (convierte una CPT en factor), `restringir`, `multiplicar`, `sumarVariable`, `reordenar`, `normalizar`, `elegirVariable` (minfill, mingrado, lista) y `eliminacionDeVariables`, que implementa el pseudocódigo de la sección 3.3 y registra cada paso.

- **Multiplicar** construye la lista de variables de la unión y recorre todas las combinaciones con un contador de base mixta; para cada una calcula el índice correspondiente en cada factor y multiplica.
- **Sumar una variable** acumula, para cada combinación de las variables restantes, los valores sobre los estados de la variable eliminada.
- **Errores explícitos:** consulta vacía o inexistente, variable repetida o simultáneamente consulta y evidencia, evidencia con estado inexistente, orden manual incompleto o con variables que no corresponden, y evidencia de probabilidad 0 («La evidencia tiene probabilidad 0…»).
- **Marginales:** el resultado incluye también la distribución marginal de cada variable consultada.
- **Nodos irrelevantes:** la aplicación no poda los nodos que no influyen en la consulta; los elimina como cualquier otra variable (su factor suma 1). Esto simplifica el código y no cambia el resultado; se muestra en los ejemplos 4 y 5.

5.3 Enumeración de referencia

`enumeracion` recorre todas las combinaciones de la red con un contador de base mixta, descarta las que contradicen la evidencia, multiplica las probabilidades de cada nodo dado el valor de sus padres y acumula en la posición de consulta. `compararConReferencia` contrasta ambos resultados; se omite si la

red tiene más de 3 millones de combinaciones.

5.4 Una consulta de principio a fin

La aplicación sigue este camino al pulsar **Ejecutar**: 1 `consulta.js` reúne consulta, evidencia y heurística; 2 `Inferencia.eliminacionDeVariables` valida la red y la consulta y calcula; 3 `compararConReferencia` calcula la enumeración y la diferencia máxima; 4 `consulta.js` dibuja la tarjeta de resultado y el detalle; 5 el evento `resultado` lo recibe el informe imprimible.

6. Parte 2 · Interfaz

Archivos: `index.html`, `css/estilos.css`, `js/main.js` y `js/interfaz/{util,grafo,editor}.js`.

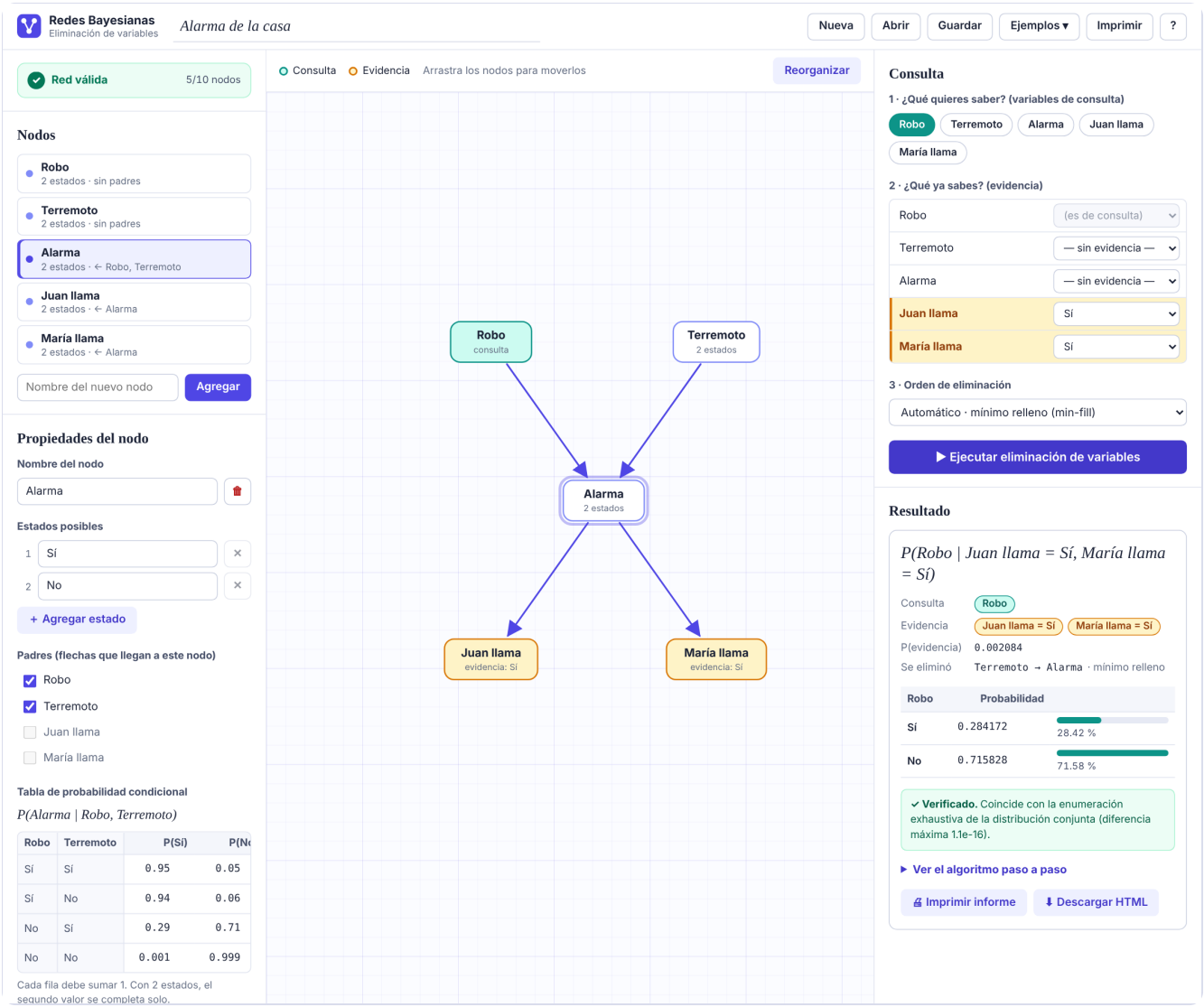


Figura 6.1. La aplicación con el Ejemplo 3 consultado.

6.1 Estructura y estilos

La página tiene tres columnas (construcción · grafo · consulta) que en pantallas estrechas se apilan. Los estilos usan variables CSS para colores y tamaños; consulta y evidencia usan colores distintos (verde azulado y ámbar), siempre acompañados de texto para no depender solo del color. Los botones y campos son controles nativos del navegador con etiquetas accesibles; el arrastre de nodos del grafo requiere ratón o pantalla táctil.

6.2 Estado global y barra superior (`main.js`)

Mantiene el objeto `app` con la red actual, el nodo seleccionado y la consulta; atiende **Nueva**, **Abrir**, **Guardar**, **Ejemplos**, **Imprimir** y **?**; pide confirmación antes de reemplazar una red con cambios; guarda un borrador en `localStorage` tras cada cambio y lo recupera al iniciar; y permite arrastrar un archivo a la ventana.

6.3 Avisos y eventos (`util.js`)

Funciones para crear elementos del DOM, mostrar avisos temporales (verde para éxito, rojo para error), ventanas de confirmación y mensajes, y el **bus de eventos** (`on` / `emit`). Los eventos son: `red` (cambió el modelo), `seleccion`, `consulta`, `posiciones` (se movió un nodo), `carga` (se abrió una red) y `resultado`.

6.4 Dibujo del grafo (`grafo.js`)

- El grafo es un **SVG**. El ancho lógico es 900 y el alto se adapta al contenido 480 a 760 ; el SVG se escala al contenedor.
- Cada nodo es un rectángulo redondeado cuyo ancho depende de la longitud del nombre. Debajo del nombre se escribe «N estados», «consulta» o «evidencia: valor».
- **Flechas:** se trazan del centro de un nodo al centro del otro, pero se recortan en el borde de cada rectángulo calculando la intersección del segmento con la caja, de modo que la punta de flecha siempre queda tocando el borde. Un marcador SVG dibuja la punta.
- **Arrastre:** eventos de puntero (ratón y táctil). Se convierte la posición de la pantalla a coordenadas del SVG, se limita a los bordes del lienzo y se emite `posiciones`; las posiciones quedan guardadas en el archivo.
- **Reorganizar** llama al acomodo automático del núcleo; los nodos nuevos se colocan en la primera cuadrícula libre para no encimarse.

6.5 Editor (`editor.js`)

- **Lista de nodos** con resumen (estados y padres) y alta de nodo nuevo.
- **Propiedades:** nombre (con borrado), estados (agregar, renombrar, quitar), padres y CPT.
- **Padres:** se muestran como casillas. Las que crearían un ciclo (los descendientes del nodo) se deshabilitan en lugar de aceptar el clic y fallar después.
- **CPT** una fila por combinación de padres, con la suma de cada fila (verde si es 1, rojo si no). Con dos estados, el segundo valor se completa solo. Botones para normalizar filas y para restablecer a uniforme.
- Cuando una acción reinicia tablas (cambio de estados o de padres), la interfaz avisa qué nodos fueron afectados.

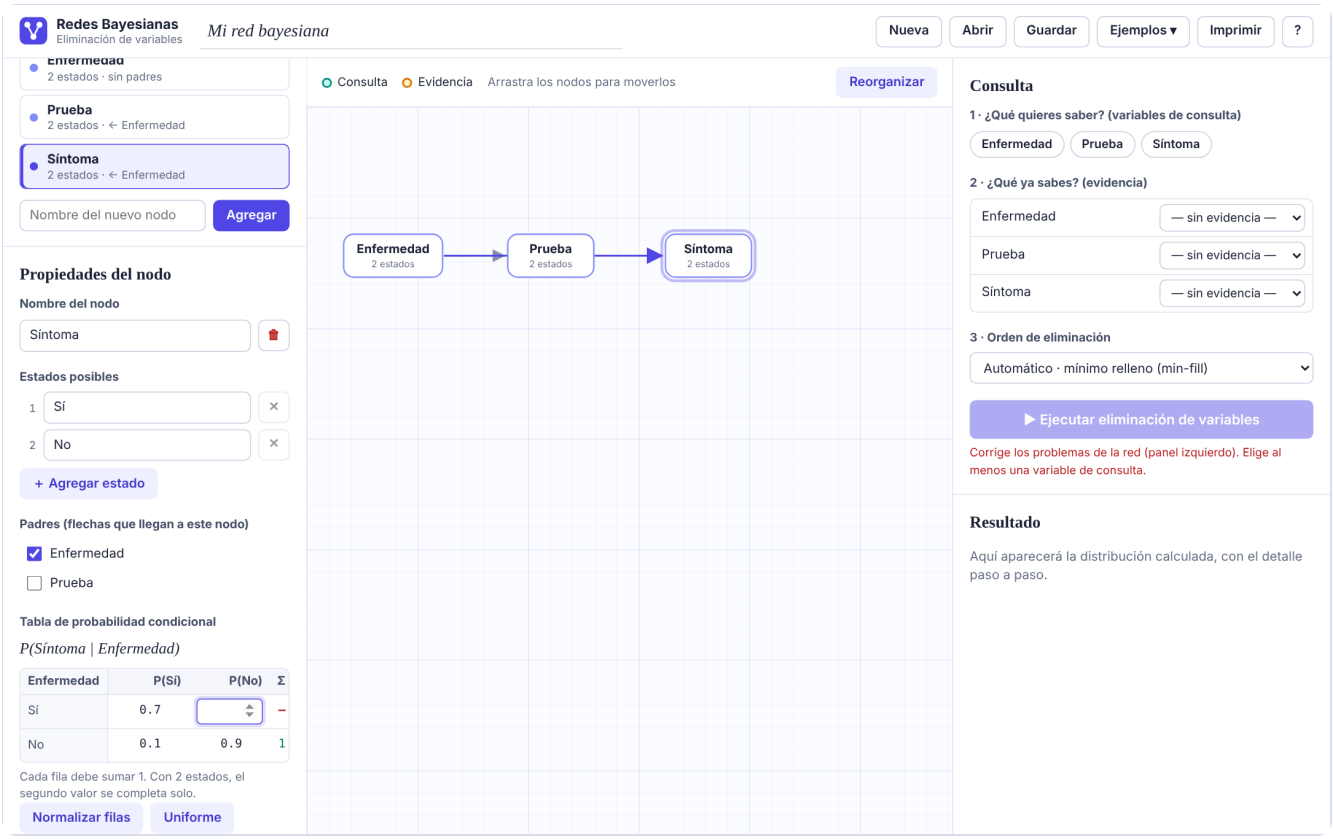


Figura 6.2. Validación en vivo: una fila que no suma 1 desactiva Ejecutar y explica el motivo.

7. Parte 3 · Consultas, archivos y salidas

Archivos: `js/interfaz/consulta.js`, `js/archivos/{almacenamiento,informe,ejemplos-data}.js`, `herramientas/generar-ejemplos.js`, `ejemplos/` y la documentación.

7.1 Panel de consultas (`consulta.js`)

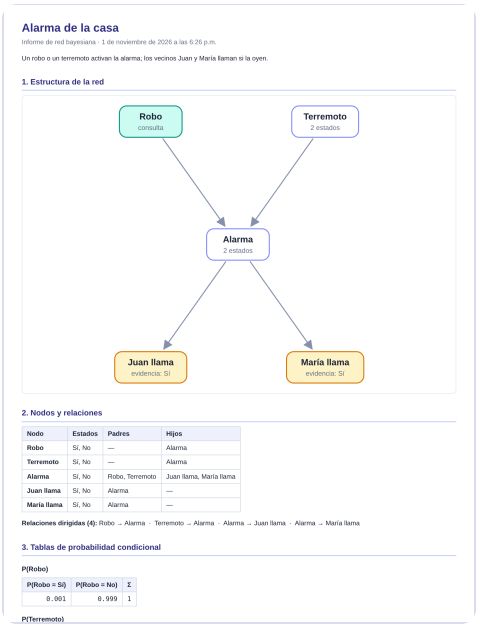
Dibuja los tres pasos (consulta, evidencia, orden). Mantiene la coherencia: una variable de consulta no puede ser a la vez evidencia (su selector se deshabilita). Antes de ejecutar muestra los motivos por los que no se puede (red inválida, sin consulta). Al ejecutar muestra: pregunta en notación P ·|·), P(evidencia), orden de eliminación, tabla con barras, recuadro de verificación y un desplegable con el algoritmo paso a paso, incluyendo cada τ.

7.2 Guardar y abrir (`almacenamiento.js`)

- **Serialización** a JSON con `formato: "red-bayesiana"` y `version: 1`; se guardan nombre, descripción, nodos con posición y una consulta sugerida.
- **Lectura con validación:** JSON mal formado, objeto que no es una red, o red con errores (ciclo, CPT inválida…) no se cargan y se muestra la lista completa de problemas.
- **Tolerancia:** campos desconocidos se ignoran; si faltan las posiciones se calculan con el acomodo automático; si falta `padres` se asume sin padres.
- **Borrador** en `localStorage` (clave `bn-borrador-v1`) con manejo de errores, ya que el navegador puede bloquearlo.

7.3 Informe imprimible (`informe.js`)

Construye una página HTML independiente con el grafo en SVG, la tabla de nodos y relaciones, todas las CPT, la consulta con su resultado y el detalle de pasos, con estilos de impresión. La ventana de impresión del navegador permite «Guardar como PDF». También se puede descargar como `.html`.



Alarma de la casa

Informe de red bayesiana · 1 de noviembre de 2026 a las 6:26 p.m.

Un robo o un terremoto activan la alarma; los vecinos Juan y María llaman si la oyen.

1. Estructura de la red

Robo
consulta

Terremoto
2 estados

Alarma
2 estados

Juan llama
evidencia: Sí

María llama
evidencia: Sí

2. Nodos y relaciones

Nodo	Estados	Padres	Hijos
Robo	Sí, No	—	Alarma
Terremoto	Sí, No	—	Alarma
Alarma	Sí, No	Robo, Terremoto	Juan llama, María llama
Juan llama	Sí, No	Alarma	—
María llama	Sí, No	Alarma	—

Relaciones dirigidas (4): Robo → Alarma · Terremoto → Alarma · Alarma → Juan llama · Alarma → María llama

3. Tablas de probabilidad condicional

P(Robo)

P(Robo = Sí)	P(Robo = No)	Σ
0.001	0.999	1

P(Terremoto)

Figura 7.1. Informe, página 1.



P(Terremoto = Sí)	P(Terremoto = No)	Σ
0.002	0.998	1

P(Alarma | Robo, Terremoto)

Robo	Terremoto	P(Alarma = Sí)	P(Alarma = No)	Σ
Sí	Sí	0.95	0.05	1
Sí	No	0.94	0.06	1
No	Sí	0.29	0.71	1
No	No	0.001	0.999	1

P(Juan llama | Alarma)

Alarma	P(Juan llama = Sí)	P(Juan llama = No)	Σ
Sí	0.9	0.1	1
No	0.05	0.95	1

P(María llama | Alarma)

Alarma	P(María llama = Sí)	P(María llama = No)	Σ
Sí	0.7	0.3	1
No	0.01	0.99	1

4. Consulta realizada

Consulta: Robo

Evidencia: Juan llama = Sí · María llama = Sí

P(evidencia) = 0.002084 · Orden de eliminación: Terremoto → Alarma

P(Robo | Juan llama, María llama)

Robo	Probabilidad	%
Sí	0.284172	28.42 %
No	0.715828	71.58 %

✓ Verificado: coincide con la enumeración exhaustiva de la distribución conjunta (diferencia máxima 1.1e-16).

Pasos del algoritmo

1. **Factores iniciales:** P(Robo) · P(Terremoto) · P(Alarma | Robo, Terremoto) · P(Juan llama | Alarma) · P(María llama | Alarma)
2. **Evidencia Juan llama = Sí:** se restringen P(Juan llama | Alarma).
3. **Evidencia María llama = Sí:** se restringen P(María llama | Alarma).
4. **Eliminar Terremoto:** se multiplican P(Terremoto) × P(Alarma | Robo, Terremoto) → φ(Terremoto, Robo, Alarma); se suma sobre Terremoto → *τ1(Robo, Alarma)*.

Robo	Alarma	valor
Sí	Sí	0.940020
Sí	No	0.059980
No	Sí	0.001578
No	No	0.998422

5. **Eliminar Alarma:** se multiplican P(Juan llama | Alarma)[Juan llama=Sí] × P(María llama | Alarma)[María llama=Sí] × τ1(Robo, Alarma) → φ(Alarma, Robo); se suma sobre Alarma → *τ2(Robo)*.

Robo	valor

Figura 7.2. Informe, página 2.

7.4 Ejemplos y herramienta generadora

`herramientas/generar-ejemplos.js` define las cinco redes en un solo lugar, las valida con el propio núcleo, calcula con la enumeración las consultas de referencia, y escribe tanto `ejemplos/*.json` y `resultados-esperados.json` como `js/archivos/ejemplos-data.js` (los ejemplos embebidos que usa el menú, para que funcionen sin servidor). Así, archivos y menú nunca se desincronizan.

8. Validación y pruebas

8.1 Estrategia

Se usaron cuatro niveles de comprobación, de menor a mayor independencia:

Cálculos hechos a mano en pruebas pequeñas (por ejemplo P Lluvia Sí | Pasto Sí) 0.16038 / 0.44838 .

Enumeración exhaustiva contra eliminación de variables, en cada consulta de la aplicación y en las pruebas.

Redes aleatorias: 300 redes generadas con una semilla fija, con consultas y evidencias al azar.

Programa de cálculo manual independiente: para este documento y el manual se escribió aparte un calculador que multiplica y suma tablas con su propia implementación, para recalcular una consulta de cada ejemplo; sus resultados se compararon con la aplicación y con la enumeración.

8.2 Pruebas automáticas

Se ejecutan con `node pruebas/pruebas.js` Node 18 o superior, sin instalar nada). Resultado: **74 de 74 pruebas correctas** (salida completa en el anexo A .

Grupo	Qué comprueba	Pruebas
A	Validaciones: mínimo y máximo de nodos, nombres repetidos, ciclos (incluido A B C A y un nodo padre de sí mismo), CPT que no suma 1, valores fuera de 0,1 o NaN, tamaño de CPT, y reinicio/renombrado al editar estados y nodos.	13
B	Eliminación de variables contra valores calculados a mano; suma 1; marginal de una raíz sin evidencia; evidencia imposible; consulta y evidencia en la misma variable; orden manual válido y rechazo de uno incompleto.	8
C	Los cinco ejemplos con tres consultas cada uno y tres órdenes de eliminación (mínimo relleno, mínimo grado y lista), contra la enumeración y los resultados esperados.	45
D	300 redes aleatorias: eliminación contra enumeración.	1
E	Guardar y abrir sin pérdidas; modificar–guardar–abrir; JSON roto; JSON que no es red; CPT inválida; ciclo; posiciones ausentes.	7
Total		**74**

8.3 Resultados de los cinco ejemplos

Ej.	Consulta	Resultado (mano = aplicación = enumeración)	P(evidencia)	Dif. máx.
1	P Lluvia | Pasto mojado Sí)	Sí 0.357688 No 0.642312	0.448380	2.8e-13

2	P Dificultad | Carta Buena)	Alta 0.261461 Baja 0.738539	0.642084	3.1e-15
3	P Robo | Juan llama Sí, María llama Sí)	Sí 0.284172 No 0.715828	0.002084	3.9e-13
4	P Batería | Arranca No, Luces = Tenues)	Buena 0.041291 Débil 0.958709	0.115739	3.6e-13
5	P Cáncer | Rayos X Positivo, Fumador Sí)	Sí 0.234079 No 0.765921	0.040700	2.2e-13

«Dif. máx.» es la mayor diferencia entre la cuenta a mano y la enumeración. Los ejemplos 1 y 3 coinciden con los valores publicados en la literatura 0.3577 y 0.284 .

8.4 Pruebas con la interfaz

Con Playwright y Chromium se recorrió automáticamente: abrir cada ejemplo, ejecutar la consulta sugerida, crear una red desde cero (agregar nodos, estados, padres, CPT , provocar los errores (nombre repetido, CPT incompleta, ciclo, archivo inválido), guardar y reabrir un archivo y comparar que quede idéntico, imprimir el informe a PDF, y revisar que no hubiera errores en la consola ni recursos que no cargaran al servir la página por HTTP. Con esas ejecuciones se tomaron las capturas del manual. Las pruebas en Windows, macOS, Firefox y Safari quedan registradas en el README como pendientes de que el equipo las realice.

9. Los cinco ejemplos

Los detalles de cada red (variables, estados, tablas, capturas y cálculo paso a paso) están en el manual de usuario, sección 7. Aquí se comparan sus características y el costo de resolverlos.

Ej.	Red	Nodos	Arcos	Combinaciones (enumeración)	Mayor factor de VE (filas)
1	Pasto mojado	3	3	8	4
2	Estudiante	4	3	24	12
3	Alarma de la casa	5	4	32	8
4	Auto que no enciende	6	5	64	4
5	Diagnóstico respiratorio	10	10	1024	8

La última columna cuenta las filas del mayor producto intermedio en la resolución a mano (con los nodos irrelevantes omitidos), con la consulta sugerida. Muestra el efecto de la eliminación: en el ejemplo de 10 nodos la enumeración recorre 1024 combinaciones, mientras que ningún factor intermedio supera unas pocas filas. Con redes tan pequeñas la diferencia de tiempo no se nota en pantalla; el beneficio crece con el tamaño de la red.

9.1 Qué aporta cada ejemplo

Ej.	Qué se quiere mostrar
1	Estructura mínima con un nodo de dos padres y un «explaining away» sencillo (aspersor y lluvia compiten como explicación del pasto mojado).
2	Variable con tres estados Calificación) y tablas de 4 filas; consulta de diagnóstico (causa a partir de efecto).
3	Estructura en forma de «diamante» (dos causas, un efecto común y dos efectos) y evidencia múltiple.
4	Nodo irrelevante Visita al taller) y evidencia en dos variables; cadena causal de tres niveles.
5	Red de 10 nodos con varias raíces, dos nodos con dos padres y ramas que no afectan la consulta Bronquitis, Disnea, Tos).

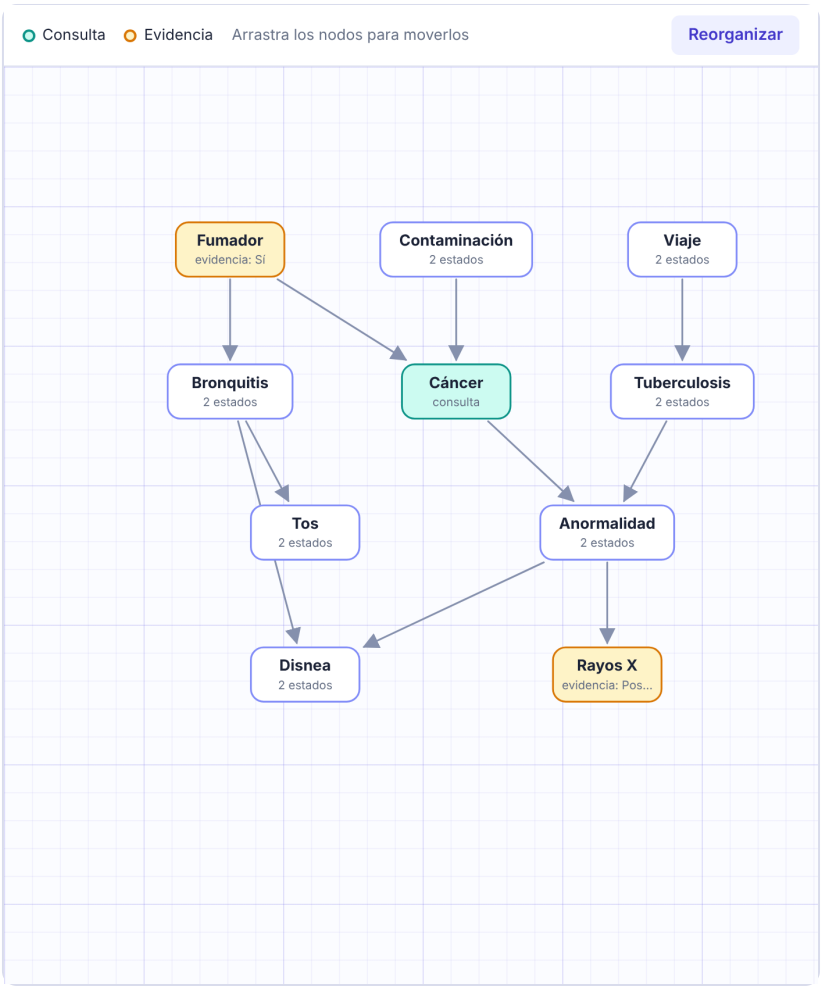


Figura 9.1. Red del Ejemplo 5 10 nodos), tal como la dibuja la aplicación.

10. Reparto de trabajo y contribuciones

10.1 Reparto por partes

El proyecto se dividió en tres partes de tamaño parecido, con límites claros entre archivos para que cada integrante pudiera trabajar y subir lo suyo sin pisar el trabajo de los demás.

Integrante	Parte	Archivos de los que es responsable
Luis Roberto Villafuerte Gómez	1 · Núcleo	`js/nucleo/red.js`, `js/nucleo/inferencia.js`, `pruebas/pruebas.js`
Luis Adrián Gómez Saavedra	2 · Interfaz	`index.html`, `css/estilos.css`, `js/main.js`, `js/interfaz/util.js`, `grafo.js`, `editor.js`
Jonathan Miguel Anguiano Huitron	3 · Consultas, archivos y salidas	`js/interfaz/consulta.js`, `js/archivos/*`, `herramientas/generar-ejemplos.js`, `ejemplos/`, `README.md`, `docs/`

10.2 Orden de subida a GitHub

Como los archivos dependen unos de otros (por ejemplo las pruebas necesitan `almacenamiento.js`), se acordó subirlos por turnos, un commit pequeño por archivo o grupo de archivos, hecho por la persona responsable de esa parte:

#	Parte	Archivos	Mensaje del commit
1	3	`README.md, .gitignore, package.json`	docs: README inicial y estructura del proyecto
2	1	`js/nucleo/red.js`	feat: modelo de la red y validaciones DAG y CPT
3	1	`js/nucleo/inferencia.js`	feat: factores y eliminación de variables
4	3	`almacenamiento.js, docs/FORMATO.md`	feat: guardar y abrir redes en JSON con validación
5	3	`generar-ejemplos.js, ejemplos/, ejemplos-data.js`	feat: cinco redes de ejemplo con resultados esperados
6	1	`pruebas/pruebas.js`	test: pruebas contra cálculos a mano y enumeración
7	2	`index.html, css/estilos.css`	feat: estructura de la página y estilos
8	2	`util.js, grafo.js`	feat: dibujo y arrastre del grafo
9	2	`editor.js`	feat: editor de nodos, estados, padres y CPT

10	3	`consulta.js`	feat: panel de consultas, evidencia y resultados
11	3	`informe.js`	feat: informe imprimible
12	2	`js/main.js`	feat: conexión de paneles y barra superior
13	3	`docs/REPARTO-Y-GIT.md`	docs: reparto de responsabilidades

10.3 Evidencia de contribuciones

La evidencia de quién hizo cada parte es el **historial de commits del repositorio**, que se puede consultar con `git shortlog -sn` (commits por persona) y `git log --stat` (archivos de cada commit). Cada integrante configuró su propio nombre y correo de Git para que GitHub atribuya los commits a la persona correcta.

10.4 Qué debe poder explicar cada integrante

Parte	Temas
1 · Núcleo	Qué es un factor y cómo se guarda; restringir, multiplicar y sumar; por qué se normaliza y qué es P(evidencia); qué cambia con el orden de eliminación; por qué la enumeración es una comprobación independiente; cómo se detecta un ciclo y se valida una CPT.
2 · Interfaz	Cómo se dibuja el grafo y se calculan las flechas; cómo funciona el arrastre; cómo se enteran los paneles de los cambios (bus de eventos); por qué no se pueden marcar padres que crean un ciclo; qué pasa con las CPT al cambiar estados o padres.
3 · Consultas, archivos y salidas	El formato del archivo y qué se valida al abrirlo; cómo se conecta el panel de consultas con el algoritmo y cómo se muestra la verificación; cómo se genera el informe; cómo se eligieron y verificaron los cinco ejemplos.

11. Limitaciones y trabajo futuro

- **Solo variables discretas** y redes de hasta 10 nodos con 2 a 8 estados, por requisito y diseño.
- **Sin poda de nodos irrelevantes.** Eliminar solo los antecesores de consulta y evidencia ahorraría trabajo en redes grandes.
- **Heurísticas voraces.** No se garantiza el orden óptimo (es NP-difícil); para estas redes el costo es mínimo.
- **Verificación por enumeración limitada** a 3 millones de combinaciones; más allá se omite el recuadro.
- **Pruebas de plataforma pendientes.** Solo se probó automáticamente en Chromium y Node.js sobre Linux; falta registrar pruebas en Windows, macOS, Firefox y Safari.
- **Mejoras posibles:** deshacer/rehacer, importar el formato estándar BIF, aprendizaje de parámetros desde datos, y evidencia «blanda» (probabilidades en lugar de valores observados).

12. Conclusiones

Se construyó una aplicación web completa que cumple los requisitos del proyecto: permite crear y validar redes bayesianas, visualizarlas, consultarlas con eliminación de variables, guardar y abrir redes, y producir salidas impresas, con cinco ejemplos y documentación. La separación entre núcleo, interfaz y archivos permitió repartir el trabajo de forma natural y probar el núcleo sin navegador.

Lo más importante es que la corrección de los resultados no se da por supuesta: cada consulta se compara con un método independiente, hay 74 pruebas automáticas (incluidas 300 redes aleatorias) y para cada ejemplo se resolvió una consulta a mano con el mismo resultado. En cuanto al aprendizaje, la eliminación de variables mostró que el orden en que se suman las variables no cambia la respuesta pero sí el tamaño de los cálculos intermedios, que es el motivo por el que existen las heurísticas de orden.

Referencias

Arnborg, S., Corneil, D. G. y Proskurowski, A. 1987 . Complexity of finding embeddings in a k-tree. *SIAM Journal on Algebraic and Discrete Methods*, 8 2 , 277 284.

Cooper, G. F. 1990 . The computational complexity of probabilistic inference using Bayesian belief networks. *Artificial Intelligence*, 42 2 3 , 393 405.

Darwiche, A. 2009 . *Modeling and Reasoning with Bayesian Networks*. Cambridge University Press.

Dechter, R. 1999 . Bucket elimination: a unifying framework for reasoning. *Artificial Intelligence*, 113 1 2 , 41 85.

Koller, D. y Friedman, N. 2009 . *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.

Lauritzen, S. L. y Spiegelhalter, D. J. 1988 . Local computations with probabilities on graphical structures and their application to expert systems. *Journal of the Royal Statistical Society, Series B*, 50 2 , 157 224.

Pearl, J. 1988 . *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann.

Russell, S. y Norvig, P. 2021 . *Artificial Intelligence: A Modern Approach* 4.ª ed.). Pearson.

Zhang, N. L. y Poole, D. 1994 . A simple approach to Bayesian network computations. *Proceedings of the 10th Canadian Conference on Artificial Intelligence*, 171 178.

Anexo A. Salida de las pruebas automáticas

`node pruebas/pruebas.js`. Para abreviar se omiten las líneas de los órdenes «mingrado» y «lista» del grupo C (cada consulta se prueba con los tres órdenes; todas pasan). El archivo completo está en `docs/evidencias/salida-pruebas.txt`.

```
A. Validaciones
  ✓ una red de 3 nodos con CPT uniformes es válida
  ✓ menos de 3 nodos no es válida
  ✓ no se pueden agregar más de 10 nodos
  ✓ nombres repetidos se rechazan (sin importar mayúsculas)
  ✓ se rechazan ciclos A→B→C→A
  ✓ un nodo no puede ser su propio padre
  ✓ un ciclo en un archivo se detecta al validar
  ✓ una fila de la CPT que no suma 1 se reporta con su descripción
  ✓ probabilidades fuera de [0,1] o NaN se rechazan
  ✓ CPT con el número de filas equivocado se rechaza
  ✓ cambiar los estados de un padre reinicia la CPT del hijo con el tamaño correcto
  ✓ eliminar un nodo limpia las referencias de sus hijos
  ✓ renombrar un nodo actualiza a sus hijos

B. Eliminación de variables vs. cálculos hechos a mano
  ✓ Pasto mojado: P(Lluvia=Sí | Pasto=Sí) = 0.16038 / 0.44838 ≈ 0.357687
  ✓ Alarma: P(Robo=Sí | Juan=Sí, María=Sí) ≈ 0.284 (valor clásico de Russell & Norvig)
  ✓ Sin evidencia, la marginal de una raíz es su propia distribución a priori
  ✓ La distribución resultante suma 1
  ✓ Evidencia imposible (probabilidad 0) produce un error claro
  ✓ Una variable no puede estar en la consulta y en la evidencia
  ✓ El orden de eliminación manual se respeta y no cambia el resultado
  ✓ Un orden manual incompleto se rechaza

C. Los 5 ejemplos: eliminación de variables vs. enumeración y resultados esperados
  ✓ 3 nodos · P(Lluvia | Pasto mojado) · minfill
  ✓ 3 nodos · P(Aspersor | Pasto mojado) · minfill
  ✓ 3 nodos · P(Pasto mojado | —) · minfill
  ✓ 4 nodos · P(Dificultad | Carta) · minfill
  ✓ 4 nodos · P(Calificación | Preparación) · minfill
  ✓ 4 nodos · P(Dificultad,Preparación | Calificación) · minfill
  ✓ 5 nodos · P(Robo | Juan llama,María llama) · minfill
  ✓ 5 nodos · P(Alarma | Juan llama) · minfill
  ✓ 5 nodos · P(Robo,Terremoto | Alarma) · minfill
  ✓ 6 nodos · P(Batería | Arranca,Luces) · minfill
  ✓ 6 nodos · P(Gasolina | Arranca) · minfill
  ✓ 6 nodos · P(Visita al taller | Frío) · minfill
  ✓ 10 nodos · P(Cáncer | Rayos X,Fumador) · minfill
  ✓ 10 nodos · P(Bronquitis,Tuberculosis | Disnea,Tos) · minfill
  ✓ 10 nodos · P(Anormalidad | Rayos X) · minfill

D. Redes aleatorias
  ✓ 300 redes aleatorias: la eliminación de variables coincide con la enumeración

E. Guardar y abrir
```

```
  ✓ los 5 ejemplos se guardan y se abren sin perder información
  ✓ modificar, guardar y abrir de nuevo conserva los cambios
  ✓ un archivo con JSON roto da un mensaje claro
  ✓ un JSON que no es una red se rechaza
  ✓ un archivo con CPT inválida se rechaza y explica por qué
  ✓ un archivo con un ciclo se rechaza
  ✓ si el archivo no trae posiciones, se calculan

74/74 pruebas correctas.
```

Anexo B. Estructura del repositorio

```
index.html                     Página principal
css/estilos.css                Estilos
js/nucleo/red.js               Modelo de la red y validaciones          (Parte 1)
js/nucleo/inferencia.js        Factores, eliminación y enumeración      (Parte 1)
js/interfaz/util.js            Avisos, ventanas, bus de eventos         (Parte 2)
js/interfaz/grafo.js           Dibujo y arrastre del grafo              (Parte 2)
js/interfaz/editor.js          Editor de nodos, estados, padres y CPT   (Parte 2)
js/main.js                     Estado global y barra superior           (Parte 2)
js/interfaz/consulta.js        Consultas y resultados                   (Parte 3)
js/archivos/almacenamiento.js  Guardar y abrir JSON                     (Parte 3)
js/archivos/informe.js         Informe imprimible                       (Parte 3)
js/archivos/ejemplos-data.js   Ejemplos embebidos (generado)            (Parte 3)
ejemplos/                      5 redes y resultados esperados
pruebas/pruebas.js             Pruebas automáticas
herramientas/generar-ejemplos.js  Define y genera los ejemplos
docs/FORMATO.md                Formato del archivo de red
docs/REPARTO-Y-GIT.md          Reparto y guía de Git
docs/Manual-de-usuario.pdf     Manual de usuario
docs/Memoria-tecnica.pdf       Este documento
docs/evidencias/               Capturas y salidas de pruebas
```

Anexo C. Evidencias

La carpeta `docs/evidencias/` contiene las capturas de pantalla reales de la aplicación (interfaz, tutorial paso a paso, los cinco ejemplos con su consulta, resultado y pasos, el informe imprimible y los mensajes de error), un ejemplo de red creada desde cero (`mi-primera-red.json`), un informe de ejemplo en PDF (`informe-ejemplo3.pdf`) y la salida de las pruebas (`salida-pruebas.txt`).



Figura C.1. Resultado del Ejemplo 3 con verificación.